Informe Proyecto II - Rutas aéreas

EIF207 Estructuras de Datos · Proyecto II · II Ciclo 2026 Universidad Nacional de Costa Rica, Sede Regional Brunca Estudiante: Karen Zeledón Alvarez (trabajo individual)

El programa es un planificador de vuelos en C++17 con interfaz gráfica en SFML 3. Carga 40 aeropuertos y 202 rutas dirigidas desde archivos de texto, y responde cuál es la ruta con menos escalas (BFS), la más rápida y la más barata (Dijkstra), qué destinos son alcanzables (DFS) y cómo cambian los resultados al cancelar rutas.

El código está en tres módulos: Grafo (estructura, carga y algoritmos), Interfaz (mapa, panel y animación) y mainConsola (prototipo usado para pruebas).

1. Estructuras de datos usadas

El grafo es dirigido y ponderado, y se representa con lista de adyacencia. Cada aeropuerto tiene un índice entero, y todas las estructuras se indexan por ese número.

Estructura	Para qué se usa	Por qué
vector<Aeropuerto>	Datos de cada aeropuerto (código, x, y, ciudad, país)	Acceso O(1) por índice
unordered_map<string,int>	Traducir un código IATA a su índice	Búsqueda promedio O(1); detecta códigos inexistentes y duplicados
vector<vector<Ruta>>	Lista de adyacencia: rutas salientes de cada aeropuerto	Memoria O(n + m); recorrer los vecinos cuesta O(grado)
struct Ruta {destino, minutos, costo}	Una arista con sus dos pesos	Un solo grafo sirve para los dos criterios de Dijkstra
queue<int>	BFS	FIFO: procesa por niveles, o sea por cantidad de vuelos
priority_queue con greater	Dijkstra	Montículo mínimo: saca el aeropuerto con menor distancia en O(log m)
Pila de llamadas (DFS recursivo)	Alcanzabilidad	Recorre en profundidad con código corto y claro
vector<bool> visitado, vector<int> padre, vector<double> dist	Estado de los recorridos	Arreglos indexados por aeropuerto, acceso O(1)
vector<RutaCancelada>	Rutas canceladas	Permite restaurarlas sin volver a leer el

Estructura	Para qué se usa	Por qué
	(origen + datos)	archivo

Cancelar una ruta la borra de la lista de adyacencia en lugar de marcarla como inactiva. Así BFS, Dijkstra y DFS no necesitan ninguna condición extra, y el recálculo es automático porque cada consulta recorre el grafo actual.

Dijkstra está implementado una sola vez. Recibe un enum Criterio { MINUTOS, COSTO }, y la función peso(ruta, criterio) decide qué valor sumar. BFS y Dijkstra devuelven el mismo tipo, ResultadoRuta (camino, totales y datos de cada tramo), que usan tanto la consola como la interfaz y la animación.

2. Complejidad de cada operación

n = cantidad de aeropuertos, m = cantidad de rutas, g = grado de salida de un aeropuerto, c = rutas canceladas.

Operación	Complejidad	Explicación
Cargar aeropuertos	O(n)	Una inserción en el vector y una en el mapa por línea
Cargar rutas	O(m)	Dos búsquedas O(1) en el mapa y un push_back por línea
Consultar aeropuerto	O(1 + g)	Buscar el índice y listar sus rutas salientes
BFS (menos escalas)	O(n + m)	Cada aeropuerto entra una vez a la cola; cada ruta se revisa una vez
Dijkstra (rápida o barata)	O((n + m) log m)	Cada relajación puede meter un par en el montículo
Reconstruir el camino	O(m) en el peor caso	Por cada tramo se busca la ruta en la lista del origen
DFS (alcanzables)	O(n + m)	Cada aeropuerto se visita una vez; cada ruta se revisa una vez
No llegan a un destino	O(n · (n + m))	Un DFS desde cada aeropuerto
Cancelar ruta	O(g)	Buscar en la lista del origen y borrar con erase
Restaurar rutas	O(c)	Un push_back por ruta cancelada
Rutas críticas	O(m · (n + m))	Por cada ruta: quitarla, hacer un DFS y devolverla

Con n = 40 y m = 202 todas las operaciones son instantáneas. Las últimas dos podrían mejorarse con el grafo invertido, que se explica en la pregunta 3.

3. Pregunta 1: rutas de DEN→SJO, LIR→SJU y SJO→MVD

En los tres viajes, cada criterio da una ruta distinta. Ninguna ruta gana en los tres criterios a la vez.

Viaje	Criterio	Ruta	Vuelos	Minutos	Costo
DEN→SJO	Menos escalas	DEN-LAX-SJO	2	500	$895
DEN→SJO	Más rápida	DEN-DFW-IAH-SJO	3	**385**	$525
DEN→SJO	Más barata	DEN-DFW-SAL-MGA-SJO	4	425	**$435**
LIR→SJU	Menos escalas	LIR-JFK-SJU	2	510	$585
LIR→SJU	Más rápida	LIR-SJO-PTY-CCS-SJU	4	**355**	$520
LIR→SJU	Más barata	LIR-SAL-MIA-SJU	3	385	**$405**
SJO→MVD	Menos escalas	SJO-MIA-BSB-GRU-MVD	4	870	$1125
SJO→MVD	Más rápida	SJO-PTY-LIM-EZE-MVD	4	**595**	$685
SJO→MVD	Más barata	SJO-PTY-LIM-GRU-MVD	4	720	**$640**

¿Por qué la ruta de menos escalas no es la más rápida? BFS cuenta aristas y no mira los minutos. En DEN→SJO elige DEN-LAX-SJO porque son solo 2 vuelos, pero el tramo LAX-SJO dura 355 minutos. La ruta por DFW e IAH agrega una escala, pero sus tres tramos suman 385 minutos. Más escalas con tramos cortos pueden sumar menos tiempo que pocos tramos largos. En SJO→MVD las tres rutas tienen 4 vuelos: BFS devuelve la primera que encuentra con 4 vuelos (870 min), que no tiene por qué ser la de menos minutos.

¿Por qué la más rápida no es la más barata? Minutos y costo son pesos independientes: una ruta corta puede ser cara. En DEN→SJO la ruta más rápida usa IAH-SJO, que cuesta $320. La más barata la evita con vuelos regionales económicos (DFW-SAL-MGA-SJO), a cambio de 40 minutos y una escala más. Dijkstra con cada peso optimiza solo ese peso, así que da rutas distintas.

4. Pregunta 2: BFS, Dijkstra y pesos negativos

¿Por qué BFS no sirve para la ruta más rápida? BFS da por terminado un aeropuerto la primera vez que lo descubre, sin comparar minutos. Eso garantiza la menor cantidad de vuelos, pero no la menor suma de pesos. En DEN→SJO, BFS marca SJO al llegar desde LAX (500 min) y nunca lo corrige. Dijkstra, en cambio, lo mejora a 385 min cuando lo alcanza desde IAH: a eso se le llama relajar una arista.

¿Qué condición garantiza que Dijkstra funcione? Que ningún peso sea negativo. Cuando un aeropuerto sale de la cola con distancia d, cualquier otro camino hacia él pasa por aeropuertos con distancia mayor o igual a d. Sumarle pesos no negativos no puede dar menos, así que d ya es definitiva. El programa lo asegura: cargarRutas rechaza el archivo si alguna ruta tiene minutos o costo negativos.

¿Y si una ruta tuviera costo negativo (un reembolso)? Se rompe esa garantía: un aeropuerto ya dado por definitivo podría abaratarse después pasando por la arista negativa, y Dijkstra no lo corregiría. El resultado podría ser incorrecto. Si además hubiera un ciclo de costo total negativo, no existiría ruta más

barata, porque dar vueltas al ciclo la abarataría sin límite. Para pesos negativos se usa Bellman-Ford, que también detecta esos ciclos, con costo O(n · m).

5. Pregunta 3: alcanzabilidad desde y hacia SJO

No alcanzable desde SJO: KIN. Kingston tiene rutas salientes (a HAV, MIA y SDQ), pero ninguna ruta del archivo llega a KIN. Por eso nadie puede llegar ahí, desde ningún aeropuerto.

No pueden llegar a SJO: ASU y MVD. ASU solo vuela a MVD y MVD solo vuela a ASU. Forman un ciclo cerrado: se puede llegar a ellos desde EZE, GRU o SCL, pero no se puede salir.

Cómo lo calcula el programa:

Alcanzables desde SJO: un DFS recursivo desde SJO marca visitado[i] = true en cada aeropuerto al que llega. Al terminar, los que siguen en false son los no alcanzables. El arreglo visitado también evita ciclos infinitos, como ASU ↔ MVD. En la interfaz, ese mismo vector colorea los puntos: celeste si es alcanzable, gris si no.

Quiénes no llegan a SJO: la función noLleganA hace un DFS desde cada uno de los otros aeropuertos y revisa si en alguno queda marcado SJO. Cuesta O(n · (n + m)).

Mejora posible: construir el grafo invertido, donde cada ruta A→B pasa a ser B→A, y hacer un solo DFS desde SJO. Lo que se alcanza en el grafo invertido son exactamente los aeropuertos que pueden llegar a SJO, y eso costaría O(n + m).

6. Pregunta 4: ruta cuya cancelación aísla un aeropuerto

La ruta es LIM→CUZ. Es la única ruta del archivo que llega a Cusco. Si se cancela, CUZ deja de ser alcanzable desde SJO. Se comprobó en la interfaz: se cancela LIM→CUZ, se consultan los alcanzables desde SJO, y CUZ aparece en gris junto a KIN.

Cómo se encontró: la función rutasCriticas calcula primero los alcanzables desde SJO con el grafo completo. Luego, para cada ruta que sale de un aeropuerto alcanzable, hace tres cosas: la quita temporalmente, ejecuta un DFS desde SJO y la devuelve a la misma posición. Si algún aeropuerto que antes era alcanzable deja de serlo, esa ruta es crítica. Se compara contra los alcanzables iniciales y no contra todos los aeropuertos, porque KIN ya era inalcanzable antes de cancelar nada.

¿Es la única? Para cancelaciones de **una sola ruta**, sí: el programa revisó las 202 rutas y solo LIM→CUZ aísla un aeropuerto. Los demás aeropuertos tienen al menos dos rutas de llegada desde aeropuertos alcanzables. Pero si se cancelan **dos** rutas a la vez aparecen otros casos. Por ejemplo, GUA solo recibe vuelos desde SAL y SAP: cancelar SAL→GUA y SAP→GUA juntas deja a GUA inalcanzable.

7. Pregunta 5: lista de adyacencia frente a matriz

Se usa lista de adyacencia porque la red es **dispersa**: cada aeropuerto tiene pocas rutas comparado con la cantidad de aeropuertos. La lista guarda solo las rutas que existen. La matriz reserva una celda para cada par posible, exista la ruta o no.

Supuestos del cálculo: cada ruta (int destino + int minutos + double costo) ocupa 16 bytes. Una celda de matriz con minutos y costo también ocupa 16 bytes. Cada vector vacío ocupa 24 bytes en compilación de 64 bits.

Red	Matriz ($n^2 \times 16$ B)	Lista ($m \times 16$ B + $n \times$ 24 B)	Celdas usadas en la matriz
n = 40, m = 202	1 600 celdas ≈ 25,6 KB	≈ 4,2 KB	12,6 %
n = 4 000, m ≈ 80 000	16 000 000 celdas ≈ 256 MB	≈ 1,4 MB	0,5 %

Con los datos del proyecto la matriz ya usa unas 6 veces más memoria. Con 4 000 aeropuertos la diferencia es de unas 180 veces, y la matriz desperdicia el 99,5 % de sus celdas.

También cambia el tiempo. Con la lista, recorrer los vecinos de un aeropuerto cuesta O(grado). Con la matriz hay que revisar una fila entera, O(n), aunque el aeropuerto tenga 5 rutas. Por eso BFS y DFS serían $O(n^2)$ con matriz y O(n + m) con lista. La matriz solo tiene ventaja para preguntar "¿existe la ruta A→B?" en O(1), y el programa casi no necesita esa operación.

8. Pregunta 6: complejidad de BFS, DFS y Dijkstra

Algoritmo	Complejidad	Por qué
BFS	O(n + m)	Cada aeropuerto se encola y desencola una vez (se marca al encolar); cada lista de adyacencia se recorre una vez
DFS	O(n + m)	dfs(u) se llama una vez por aeropuerto; cada ruta se revisa una vez
Dijkstra con priority_queue	O((n + m) log m)	Hasta m inserciones y extracciones en el montículo, cada una O(log m)

Como $m \le n^2$, $\log m \le 2 \log n$, así que la cota de Dijkstra equivale a O((n + m) log n).

Qué cambia con la cola de prioridad. Sin ella, en cada paso habría que recorrer los n aeropuertos para encontrar el no definitivo con menor distancia. Eso cuesta O(n) por paso y $O(n^2 + m)$ en total. Con el montículo mínimo, sacar el menor cuesta O(log m). En una red dispersa como esta (m mucho menor que n^2) el montículo es mejor: con n = 4 000 y m = 80 000 son unos 1,4 millones de operaciones contra unos 16 millones. La versión lineal solo convendría en un grafo muy denso, con m cerca de n^2.

priority_queue no permite bajar la prioridad de un elemento que ya está adentro. Por eso, cuando un aeropuerto mejora, se inserta un par nuevo y el viejo se descarta al salir con if (d > dist[u]) continue; (eliminación perezosa). Esa es la razón de que el montículo pueda tener hasta m elementos y de que la cota se exprese con log m.